Super-Stretch Ideas

Beyond Level 3: a curiosity list, not an assignment

VIDEO GAME CREATION

What This Is

Every lesson's Level 3 is a short, deliberately achievable menu of stretch goals - all of them worked out and confirmed possible in every supported engine before they ever reach a Handout. This document is different. It's a longer, generic list of "what else is out there" ideas for a student who finishes Level 3 early, or who's just curious what game development looks like beyond this week's exercise.

A few important differences from Level 3:

- Nothing here is required, tracked, or graded. It's fine - expected, even - for a student to read this list and try nothing on it.
- These ideas are written generically, not for a specific engine, and not against this lesson's exact Key Press Reactor exercise. Some will be easy in one engine and awkward or impossible in another; some describe concepts this course covers properly in a later lesson; some may need looking things up outside class.
- Unlike the core lesson materials, self-containment doesn't apply here - a student chasing one of these ideas is welcome (encouraged, even) to search the engine's own documentation, forums, or tutorials. That kind of independent digging is the point.
- If an idea looks interesting but the how isn't obvious, that's a good reason to ask - in class, or by flagging it for a future lesson.

Movement & Feel

- **Ease in and out:** Instead of snapping to full speed instantly, ramp up to it (acceleration) and slow back down when a key is released (deceleration/friction), instead of stopping dead.
- **Normalize diagonal movement:** If two direction keys are held at once, moving diagonally shouldn't be faster than moving in a single direction - scale the combined movement back down to match.
- **Bounce instead of stop:** At the screen edge, instead of just clamping in place, reverse direction (or velocity) so the character bounces off the edge.

- **Dash or sprint:** Add a second key (like Shift) that briefly boosts speed, ideally with a cooldown so it can't just be held down permanently.
- **Gravity and a jump:** Instead of free movement in all 4 directions, try a side-view character that's pulled downward by gravity and has a single jump button - the beginning of a platformer.
- **Camera follows the character:** Instead of a fixed view, have the camera/viewport scroll to keep the character in frame as it moves - useful once a level is bigger than one screen.

Visual Polish ("Juice")

- **True in-engine color tinting:** This lesson's Recolor goal swaps in a whole separate pre-made colored image for each direction, since most engines' quick tint options (image_blend, modulate, spriteRenderer.color, Tint color, and similar) only give a clean, true color on plain white/grayscale art - tint a colored sprite and the result comes out muddy, a multiply of the two colors rather than a clean swap. If you're curious why, try it yourself: duplicate your character sprite, fill it solid white, and tint that instead of swapping images - you should get a clean, exact color for every direction with a single sprite, no swapping needed. Worth knowing which engines can do this cleanly and which can't, and why TIC-80 is the one exception in this lesson that never had the problem to begin with (see its Recolor notes).
- **Face the direction of movement:** Flip or mirror the character horizontally (or rotate it) so it visually faces the direction it's currently moving.
- **Squash and stretch:** Briefly scale the character slightly wider/shorter when it starts moving and slightly taller/thinner when it stops, for a springy, less rigid feel.
- **Trail or particle effect:** Spawn a fading trail, dust puff, or sparkle effect while the character is in motion.
- **Screen shake:** Briefly jitter the camera or view for an impact moment, e.g. when hitting the screen edge.
- **Fade in or out:** Have the character (or the whole scene) fade in at the start, or fade out when triggered, instead of just appearing/disappearing instantly.

Audio

- **Footsteps synced to animation:** Instead of one looping tone, trigger a short sound on each animation-frame change, so it reads as footsteps rather than a siren.
- **Background music:** Add a simple looping music track that plays for the whole scene, separate from the movement sound effect.
- **Vary pitch or volume:** Slightly randomize a repeated sound effect's pitch or volume each time it plays so it doesn't feel robotic on repeat.

Game State & Feedback

- **On-screen counter:** Display a running count of something - button presses, steps taken, seconds survived - updating live as text on screen.

- **Simple timer:** Start a countdown or count-up timer when the scene begins and display it.
- **Tunable variables:** Expose a value like speed as a named variable you (or the engine's own inspector/editor UI) can tweak without digging back into the logic - useful for quickly trying different "feels."

Input & Control

- **A second player:** Add a second character controlled by a different set of keys (e.g. WASD alongside the arrow keys), sharing the same screen.
- **Gamepad support:** Get the same movement working from a connected controller, alongside (not instead of) the keyboard.
- **On-screen touch controls:** Add clickable/tappable on-screen buttons or a virtual joystick as an alternative to keyboard input.
- **True press-and-hold movement (001 Game Creator):** This lesson's 001 Game Creator path moves the character once per key press, since Input Triggers only fire on Pressed/Released, not while a key is held. For movement that continues the whole time a key is down, use the Change Movement X / Change Movement Y events instead - set them to 1 or -1 on Pressed and back to 0 on Released, and the engine keeps applying that movement every frame on its own using the actor's Speed and Acceleration, no loop required. A nice look at how a single engine can solve "hold to move" a completely different way than a polling loop does.
- **Remappable controls (Defined Control):** Instead of binding a script straight to one physical key, some engines let you define a named control - like "Move Right" - and then attach several different inputs to it (arrow key, WASD key, a gamepad button) so your scripts never have to change no matter which one the player uses. 001 Game Creator's Input Triggers have exactly this via their "Defined Control" option; look for an equivalent remapping system in whichever engine you're using.
- **Reading multiple keys reliably at once (Scratch):** Scratch's "when key pressed" scripts repeat by riding your operating system's own keyboard auto-repeat feature - the same mechanism that repeats a letter when you hold it down in a text box. Most operating systems only actively auto-repeat one key at a time, so pressing a second key (like Space to fire) while a movement key is already held can interrupt the repeat stream for the first key, making movement pause until it's pressed again. The fix: poll real key state instead of reacting to key-press events, using the Sensing category's "key ( ) pressed?" reporter inside a "forever" loop - since that reads whether a key is currently held down once per frame, it isn't affected by the operating system's one-key-at-a-time auto-repeat limit at all.

Advanced / A Peek Ahead

- **A wandering or chasing companion:** Add a second sprite that moves on its own - wandering randomly, or moving toward/away from the player character - an early taste of simple AI.
- **Remember state between runs:** Save something (like the character's last position, or a high score) so it's still there the next time the project runs.

- **Randomize on start:** Have the character's color, size, or starting position be chosen randomly each time the project runs, instead of always the same.
- **Production-grade input systems:** Some engines offer a more advanced, remappable input system (e.g. Unreal's Enhanced Input, Unity's Input System package) as an alternative to the simplest built-in key checks - worth a look for anyone curious how bigger, shipped games handle input.

A Note for the Curious

None of this is a checklist to work through in order, and it's genuinely fine if nothing here appeals yet - most of it will make a lot more sense, and be a lot easier to build, after a few more lessons. It's here so that "what else could I try?" always has an answer, even on a week where Level 3 gets finished with time to spare.